THE ART OF READABLE CODE · CHAPTER 8

Breaking Down Giant Expressions

Code that comes in chunks too big to swallow is hard to understand. Here is how to cut it into digestible pieces.

Team Members

Chou Kosol
Kem Sreyoun
Hang Muykhorng
Leng Vicheka

Dustin Boswell & Trevor Foucher · O'Reilly · Chapter summary

Why big expressions hurt

The giant squid has a donut-shaped brain wrapped around its esophagus. Swallow too much at once and it gets brain damage.

Code works the same way. Research suggests we can hold only **three or four “things”** in mind at once. The larger the expression, the harder it is to understand.

KEY IDEA

Break down your giant expressions into more digestible pieces.

Technique 1: Explaining variables

Introduce an extra variable that captures a subexpression. Its name explains what the piece means.

BEFORE

```
if line.split(':')[0].strip() == "root":
    ...
```

AFTER

```
username = line.split(':')[0].strip()
if username == "root":
    ...
```

Technique 2: Summary variables

Even when an expression is understandable, replace it with a short name you can think about more easily.

request.user.id == document.owner_id has five variables. The real concept is: **does the user own the document?**

Defining it at the top also signals: “this concept is used throughout the function.”

```
final boolean user_owns_document =
  (request.user.id == document.owner_id);

if (user_owns_document) {
  // user can edit this document...
}
if (!user_owns_document) {
  // document is read-only...
}
```

Technique 3: De Morgan's laws

```
not (a or b or c)
⇔ (not a) and (not b) and (not c)
```

```
not (a and b and c)
⇔ (not a) or (not b) or (not c)
```

Rule of thumb: **"distribute the not and switch and/or."** Use it to make a boolean condition read cleanly.

```
if (!(file_exists && !is_protected)) Error("Sorry, could not read file.");
if (!file_exists || is_protected) Error("Sorry, could not read file.");
```

Beware of abusing short-circuit logic

CLEVER · ONE LINE

```
assert((!(bucket = FindBucket(key))) || !bucket->IsOccupied());
```

CLEAR · TWO LINES

```
bucket = FindBucket(key);
if (bucket != NULL) assert(!bucket->IsOccupied());
```

Key idea

“Clever” nuggets of code are often confusing when others read them later.

Still fine

if (object && object->method()) and x = a || b || c

Example: overlaps_with() spirals

```
return (other.begin <= self.begin <= other.end or
        other.begin <= self.end <= other.end)
```

ATTEMPT 1 · WRONG

Range(0, 2).overlaps_with(Range(2, 4)) returns True, but they only touch at 2.

```
return (other.begin <= self.begin < other.end or
        other.begin < self.end <= other.end)
```

ATTEMPT 2 · WRONG

Range(0, 10).overlaps_with(Range(3, 5)) returns False, but the first range surrounds the second.

```
return (other.begin <= self.begin < other.end or
        other.begin < self.end <= other.end or
        (self.begin <= other.begin and self.end >= other.end))
```

ATTEMPT 3 · CORRECT

Right for every case, but six comparisons. Nobody can read it and feel confident.

The signal: a simple problem turned convoluted. There is probably an easier way.

Solve the opposite problem

“Doesn’t overlap” has only two cases:

1. The other range ends before this one begins.
2. The other range begins after this one ends.

Each line now has one comparison, leaving the reader enough brainpower to check whether <= is right.

```
bool Range::OverlapsWith(Range other) {
  if (other.end <= begin) return false;
  // They end before we begin
  if (other.begin >= end) return false;
  // They begin after we end

  return true;
  // Only possibility left: they overlap
}
```

Same tools for giant statements

Pull repeats into summary variables (DRY)

```
var thumbs_up = $("#thumbs_up" + n);
var thumbs_down = $("#thumbs_down" + n);
var vote_value = $("#vote_value" + n).html();
var hi = "highlighted";
```

Six copies of “highlighted” became one, which exposed a typo (“highighted”) hiding in the original, shortened lines, and left one place to change.

Collapse a repeated pattern

```
#define ADD_FIELD(field) add_to->set_##field(
  add_from.field() + add_to->field())
ADD_FIELD(total_memory);
ADD_FIELD(free_memory);
```

Lines that were long and almost-identical now show their essence at a glance. Use macros sparingly, but here they help.

Takeaways

Name the pieces

Explaining variables split the expression, document it, and surface the main concepts.

Rewrite the logic

De Morgan’s laws turn !(a && !b) into !a || b.

Two values per if

Aim for conditions with no more than two values. If that seems impossible, negate the problem.

Be aggressive

The same techniques apply to whole blocks of code. Skip “clever” one-liners.